EDUSHIELD

Child Education Insurance Management System

14 Workflows	Spring Boot 3.x	Enterprise Grade

SYSTEM WORKFLOW DOCUMENTATION

End-to-end technical reference documenting all operational flows, actor interactions, state transitions, and integration patterns.

WORKFLOWS	ACTORS	API ENDPOINTS	STATE MACHINES
14 Flows	5 Roles	22+ APIs	3 Lifecycles

PREPARED BY
Praveen Vankudoth

DATE
March 18, 2026

VERSION
1.0.0

CLASSIFICATION
CONFIDENTIAL

Table of Contents

1. Introduction to System Workflows....4
- 1.1 Document Purpose....4
- 1.2 System Architecture Context....4
- 1.3 Workflow Inventory....4
- 1.4 Common Conventions....5

2. Workflow Overview....5

3. User Authentication Workflow....6
- 3.1 Local Registration Sub-flow....6
- 3.2 Google OAuth2 Sub-flow....7
- 3.3 Login and JWT Issuance Sub-flow....7
- 3.4 State Transitions....7

4. Policy Discovery and Selection Workflow....8

5. Policy Application Workflow....9
- 5.1 Step-by-Step Process....9
- 5.2 State Transitions....10

6. Risk Assessment Workflow....11
- 6.1 Scoring Model Detail....11
- 6.2 Multiplier Logic....12

7. Underwriting Approval Workflow....13
- 7.1 Approval Sub-flow....13
- 7.2 Rejection Sub-flow....14
- 7.3 State Transitions....14

8. Policy Activation and Premium Payment Workflow....15
- 8.1 First Payment Activation....15
- 8.2 Recurring Payment....16
- 8.3 State Transitions....16

9. Claim Submission Workflow....17

10. Claim Verification and Approval Workflow....18
- 10.1 Verification Sub-flow....18
- 10.2 Approval Sub-flow....19
- 10.3 State Transitions....19

11. Document Upload and Verification Workflow....20

12. Email Notification Workflow....21

13. Audit Logging Workflow....22

14. Admin Management Workflow....23
- 14.1 User Management....23
- 14.2 Policy Management....24
- 14.3 Dashboard Analytics....24

15. Staff Management Workflow....25

16. System Integration Workflow.....26
16.1 Inter-Service Communication.....26
16.2 Full End-to-End Flow.....27

1 Introduction to System Workflows

1.1 Document Purpose

This System Workflow Documentation provides a complete, authoritative reference for all operational flows within the EduShield Child Education Insurance Management System. It is intended for backend developers, frontend engineers, quality assurance teams, business analysts, and system architects who require a precise understanding of how each business process executes from trigger to completion.

Each workflow documented herein maps directly to the Spring Boot 3.x backend implementation, reflecting actual service classes, API endpoints, JPA repository interactions, and state machine transitions. The document serves as both a design reference and an onboarding guide for new team members.

1.2 System Architecture Context

EduShield is implemented as a modular monolithic Spring Boot application with a layered architecture: Controller → Service → Repository → Database. The Angular 17/18 single-page application communicates exclusively through the RESTful API layer. All security controls are enforced at the API boundary using Spring Security with JWT Bearer token validation.

The system manages three primary lifecycle state machines in parallel: the Policy Application lifecycle, the Policy Subscription lifecycle, and the Claim lifecycle. These state machines are strictly enforced — no state may be bypassed or transitioned out-of-sequence.

1.3 Workflow Inventory

No.	Workflow Name	Primary Actor	Criticality
1	User Registration and Authentication	Customer / Guest	Critical — Entry point for all users
2	Policy Discovery and Selection	Customer / Guest	High — Precedes all applications
3	Policy Application	Customer	Critical — Core business transaction
4	Risk Assessment	System (Automated)	Critical — Determines premium and eligibility
5	Underwriting Approval	Underwriter / Admin	Critical — Authorizes policy issuance
6	Policy Activation and Premium Payment	Customer	Critical — Activates coverage
7	Claim Submission	Customer	High — Initiates benefit disbursement
8	Claim Verification and Approval	Claims Officer	Critical — Authorizes payout
9	Document Upload and Verification	Customer / Underwriter	High — Supports decisions
10	Email Notification	System (Async)	Medium — Customer communication

11	Audit Logging	System (Background)	High — Compliance and forensics
12	Admin Management	Admin	High — Platform governance
13	Staff Management	Admin	Medium — Team provisioning
14	System Integration and Analytics	Admin / System	Medium — Business intelligence

1.4 Common Conventions

Convention	Meaning
▶️ **Decision Point**	A branching point in the workflow where different paths may be followed based on data or role.
⚠️ **Exception**	An error or failure condition that terminates or redirects the normal flow.
✔️ **Rule**	A mandatory business or validation constraint that must be satisfied for the flow to proceed.
State: NAME	Represents the current status of an entity (e.g., application, claim) within its lifecycle.

2 Workflow Overview

The following matrix provides a high-level cross-reference of all workflows, showing how actors, APIs, and system state machines interconnect. EduShield's operational model can be understood as three parallel tracks running concurrently: the Customer Track, the Staff Track, and the System Track.

Workflow	Customer	Underwriter	Claims Officer	Admin / System
Authentication	✓	(Staff login)	(Staff login)	(Admin login)
Policy Discovery	✓	–	–	✓ (configure)
Policy Application	✓	✓ (review)	–	✓ (monitor)
Risk Assessment	– (auto)	Views result	–	Views result
Underwriting	–	✓	–	✓
Premium Payment	✓	–	–	✓ (monitor)
Claim Submission	✓	–	–	✓ (monitor)
Claim Approval	–	–	✓	✓
Document Upload	✓	✓ (download)	✓ (download)	✓
Email Notification	Receives	Receives	Receives	Monitors
Audit Logging	–	–	–	✓ (view only)
Admin Management	–	–	–	✓
Staff Management	–	–	–	✓ (create)
Analytics	–	–	–	✓

3 User Authentication Workflow

Workflow	User Registration and Authentication
Actors	Guest User, Customer, Underwriter, Claims Officer, Admin
Objective	Securely onboard new users and authenticate existing users, issuing a signed JWT Bearer token for all subsequent API calls.
Trigger	User navigates to the Sign Up or Login page on the Angular frontend.
APIs Involved	POST /api/auth/register, POST /api/auth/login, POST /api/auth/google, POST /api/auth/forgot-password, POST /api/auth/reset-password
Backend Services	AuthServiceImpl, GoogleAuthServiceImpl, JwtTokenProvider, EmailServiceImpl
Database Tables	SELECT / INSERT on users table; SELECT on roles table
Initial State	No active session. User is unauthenticated.
Final State	User possesses a valid JWT Bearer token. Session is active.

3.1 Local Registration Sub-flow

The local registration path handles customers who create an account using their email address and a password.

Step 1: The Angular RegisterComponent submits a POST /api/auth/register request with the payload: { name, email, password, phone (optional), role }.

Step 2: AuthController receives the request and passes it to AuthServiceImpl.register().

Step 3: AuthServiceImpl queries UserRepository.existsByEmail(email). If a record exists, an exception is thrown immediately.

▶ **Decision Point:** Does the email already exist in the users table? YES → return HTTP 400 with ERR_AUTH_05. NO → proceed.

Step 4: BCryptPasswordEncoder encodes the raw password with a work factor of 12. The plaintext password is never persisted.

Step 5: The requested role is validated against the RoleRepository. If the role string is not a recognized enum value, HTTP 400 is returned.

Step 6: A new User entity is constructed with: encodedPassword, resolvedRole, isActive = true, createdAt = now().

Step 7: UserRepository.save(user) persists the record. An audit log entry is created for the CREATE_USER event.

Step 8: JwtTokenProvider.generateToken(user) signs a JWT containing userId, email, and role with the configured HMAC-SHA256 secret.

Step 9: AuthResponse is returned: { token, type: Bearer, userId, role, email }.

3.2 Google OAuth2 Sub-flow

Step 1: User clicks the Google Sign-In button on the Angular frontend. Google's OAuth2 flow returns a Google ID Token to the Angular application.

Step 2: Angular sends POST /api/auth/google with the Google ID Token in the request body.

Step 3: GoogleAuthServiceImpl.authenticateWithGoogle(idToken) is invoked.

Step 4: GoogleIdTokenVerifier verifies the token's signature and expiry against Google's public keys.

▶️ **Decision Point:** Is the token valid? NO → HTTP 401. YES → extract email and name from the token payload.

Step 5: UserRepository.findByEmail(email) checks if an account exists.

▶️ **Decision Point:** Does the account exist? NO → create a new User with provider = GOOGLE, isActive = true, and a system-generated password hash. YES → load existing user.

Step 6: JwtTokenProvider generates a system JWT (identical structure to local login). AuthResponse returned.

3.3 Login and JWT Issuance Sub-flow

Step 1: User submits POST /api/auth/login with { email, password }.

Step 2: Spring Security's AuthenticationManager.authenticate() is invoked with a UsernamePasswordAuthenticationToken.

Step 3: UserDetailsServiceImpl loads the UserDetails for the supplied email. If not found, UsernameNotFoundException is raised.

▶️ **Decision Point:** Is the account active (isActive = true)? NO → HTTP 403. YES → proceed.

Step 4: BCryptPasswordEncoder.matches(rawPassword, storedHash) performs the verification.

▶️ **Decision Point:** Do credentials match? NO → increment failedLoginCount. If count reaches 5, lock account. Return HTTP 401. YES → reset failedLoginCount to 0.

Step 5: JwtTokenProvider.generateToken() produces a signed JWT valid for 24 hours.

Step 6: AuthResponse returned to client. Angular stores the token and attaches it as Authorization: Bearer <token> to all subsequent requests.

3.4 Validation Rules and Business Rules

Rule	Specification
Email Uniqueness	Each email address must be globally unique. Checked via UserRepository.existsByEmail() before insertion.
Password Minimum	Minimum 6 characters. Validated via @Size(min=6) on the RegisterRequest DTO.
Role Assignment	Role must be a valid system enum. Only ADMIN may provision staff roles (UNDERWRITER, CLAIMS_OFFICER).
Account Lockout	Five consecutive failed login attempts trigger account lockout (isActive set to false). Reset requires password recovery.
JWT Expiry	Tokens are valid for 24 hours. Upon expiry, the client must re-authenticate. No refresh token in v1.0.

3.5 State Transitions

From State	Event	To State	Condition / Rule
UNAUTHENTICATED	Successful registration	**JWT ACTIVE**	New user created, JWT issued.

UNAUTHENTICATED	Successful login	**JWT ACTIVE**	Credentials verified, JWT issued.
JWT ACTIVE	Token expires (24h)	**UNAUTHENTICATED**	Client must re-authenticate.
ACTIVE ACCOUNT	5 failed logins	**LOCKED**	isActive set to false; login blocked.
LOCKED	Password reset via OTP	**ACTIVE ACCOUNT**	isActive restored after successful reset.

4 Policy Discovery and Selection Workflow

Workflow	Policy Discovery and Selection
Actors	Guest User, Customer
Objective	Allow any visitor to browse available insurance products and view their details before committing to an application.
Trigger	User navigates to the Policies section in the Angular SPA.
APIs Involved	GET /api/policies, GET /api/policies/{policyId}
Backend Services	PolicyServiceImpl
Database Tables	SELECT on policy_types where is_active = true
Initial State	No policy selected. User is browsing.
Final State	User has selected a specific policy and is ready to initiate an application.

4.1 Step-by-Step Process

Step 1: The Angular PolicyListComponent initializes and dispatches GET /api/policies with optional pagination parameters (page, size).

Step 2: PolicyController.getAllActivePolicies() delegates to PolicyServiceImpl.findAllActive().

Step 3: PolicyRepository.findAllByIsActiveTrue() executes SELECT * FROM policy_types WHERE is_active = true ordered by policyName. Only live, non-discontinued products are returned.

Step 4: The response array is returned to the frontend. Each entry includes: policyId, policyName, basePremium, durationYears, minChildAge, maxChildAge, riskCoverageAmount.

Step 5: User selects a policy card. The Angular router navigates to /policies/:id.

Step 6: PolicyDetailComponent dispatches GET /api/policies/{policyId} to load the full specification.

Step 7: PolicyServiceImpl.findById(policyId) is called. If no record exists, ResourceNotFoundException is thrown, returning HTTP 404.

Step 8: The detail view renders all parameters. The frontend displays a simulated premium estimate using basePremium. A call-to-action button (Apply Now) is rendered for authenticated users.

▶️ **Decision Point:** Is the user authenticated? NO → Apply Now redirects to the login page with the policy pre-selected in session state. YES → Application wizard is launched.

4.2 Business Rules and Validation

✔️ **Rule:** Only policies with isActive = true are returned by the catalog endpoint. Discontinued policies are invisible to customers.

✔️ **Rule:** The policy detail endpoint returns any policy by ID (including inactive) to support certificate lookups by staff, but the Apply Now action is blocked for inactive policies at the service layer.

⚠️ **Exception:** If the policy catalog is empty (no active products), an empty array is returned with HTTP 200. The frontend renders an informational message.

5 Policy Application Workflow

Workflow	Policy Application
Actors	Customer, System (Risk Engine)
Objective	Allow a customer to submit an insurance application for a child beneficiary, capturing all actuarial inputs required for risk assessment and underwriting.
Trigger	Customer clicks Apply Now on a selected policy and completes the multi-step application wizard.
APIs Involved	GET /api/children/my, POST /api/policy-applications, POST /api/documents/upload
Backend Services	PolicyApplicationServiceImpl, RiskAssessmentServiceImpl, EmailServiceImpl, AuditLogServiceImpl
Database Tables	SELECT children; SELECT policy_types; INSERT policy_application; INSERT nominee; INSERT audit_logs
Initial State	Customer is authenticated. Child profile exists. Policy is active.
Final State	Application saved with status PENDING. Risk score and adjusted premium computed and stored.

5.1 Step-by-Step Process

Step 1: ApplicationWizardComponent (Angular MatStepper) guides the customer through four steps: (a) Policy confirmation, (b) Child selection, (c) Applicant financial details, (d) Document upload.

Step 2: Customer selects a child from their registered profiles via GET /api/children/my. If no child profiles exist, the wizard prompts the customer to create one first.

Step 3: Customer fills the financial detail form: annualIncome, occupation, parentAge, coverageAmount, paymentFrequency (MONTHLY/YEARLY), startDate, endDate.

Step 4: Frontend dispatches POST /api/policy-applications with the PolicyApplicationRequest payload.

Step 5: PolicyApplicationServiceImpl.applyForPolicy() is invoked. The following validations are executed sequentially:

✔ **Rule:** Policy must exist and have isActive = true. If inactive, HTTP 400 is returned.

✔ **Rule:** Child's calculated age (from dateOfBirth) must be within policy.minChildAge and policy.maxChildAge.

✔ **Rule:** annualIncome must be at least 100,000 (minimum financial stability threshold).

✔ **Rule:** coverageAmount must not exceed 5 * annualIncome. Violation returns HTTP 422 ERR_RISK_03.

✔ **Rule:** No existing application in PENDING or ACTIVE state for the same child + policy combination. Violation returns HTTP 400 ERR_APP_02.

Step 6: All validations pass. PolicyApplicationServiceImpl calls RiskAssessmentServiceImpl.calculateRisk(request). The risk score and risk category are returned synchronously.

Step 7: The calculated premium is set: calculatedPremium = policy.basePremium * riskMultiplier(riskCategory).

Step 8: A PolicyApplication entity is constructed with status = PENDING, all inputs, riskScore, riskCategory, and calculatedPremium. policyNumber is generated in the format POL-XXXXXX.

Step 9: PolicyApplicationRepository.save(application) persists the record. A Nominee record is inserted for the child beneficiary.

Step 10: AuditLogServiceImpl.logMetadata() records a CREATE_APPLICATION event.

Step 11: EmailServiceImpl.sendApplicationAcknowledgement() dispatches a confirmation email to the customer asynchronously.

Step 12: The API returns: { applicationId, policyNumber, status: PENDING, riskScore, riskCategory, calculatedPremium }.

5.2 Exception Handling

Exception	HTTP Code	Recovery
Duplicate application	400 ERR_APP_02	Customer directed to view existing application.
Coverage exceeds 5x income	422 ERR_RISK_03	Frontend suggests a lower coverage tier.
Child age out of range	400 ERR_APP_04	Customer advised to select an eligible policy.
Policy inactive	400 ERR_RISK_01	Customer redirected to the policy catalog.

5.3 State Transitions

From State	Event	To State	Condition / Rule
(None)	Customer submits application	**PENDING**	Validations passed; risk score computed; application saved.
PENDING	Customer cancels	**CANCELLED**	Customer action via PUT /cancel. Only permitted from PENDING.
PENDING	Underwriter approves	**APPROVED**	Proceeds to underwriting workflow.
PENDING	Underwriter rejects	**REJECTED**	Rejection reason recorded; customer notified.

6 Risk Assessment Workflow

Workflow	**Automated Risk Assessment**
Actors	System (called by PolicyApplicationServiceImpl)
Objective	Quantify the actuarial risk of an applicant using a deterministic, multi-factor scoring model and return a risk category and premium multiplier.
Trigger	Invoked synchronously by PolicyApplicationServiceImpl.applyForPolicy() immediately after input validation passes.
APIs Involved	Internal service call — no direct external API endpoint.
Backend Services	RiskAssessmentServiceImpl
Database Tables	No database write during calculation; results returned to caller for persistence with the application record.
Initial State	PolicyApplicationRequest received with parentAge, occupation, annualIncome, coverageAmount, basePremium.
Final State	RiskAssessmentResult returned: { riskScore (0-100), riskCategory, premiumMultiplier }.

6.1 Scoring Model Detail

6.1.1 Age Risk Component

Age Band	Score Added	Cumulative Range	Rationale
21 – 30 years	+5 points	0 – 5	Lowest statistical mortality risk cohort.
31 – 40 years	+10 points	5 – 15	Moderate age; slight life risk elevation.
41 – 50 years	+18 points	5 – 23	Elevated mortality and morbidity profile.
51 years and above	+25 points	5 – 30	Highest age-related risk band.

6.1.2 Occupation Risk Component

Occupation Category	Score Added	Example Occupations
Professional / Low Hazard	+5 points	Doctor, Engineer, Teacher, Accountant.
Moderate Hazard	+12 points	Driver, Security Personnel, Retail Worker.
High Hazard	+18 points	Construction Worker, Builder, Electrician.
Very High Hazard	+25 points	Miner, Industrial Plant Operator.

6.1.3 Financial Risk Components

Condition	Score Added	Description
Monthly Premium > 30% of Monthly Income	+20 points	Affordability risk: the premium constitutes an unsustainable proportion of income.
Coverage Amount > 20x Annual Income	+30 points	Over-insurance signal: the requested coverage far exceeds the economic loss justification.

6.2 Multiplier Logic and Category Assignment

Once all component scores are summed, RiskAssessmentServiceImpl maps the total to a risk category and a corresponding premium multiplier. The formula for the final monthly premium is: Final Premium = Base Premium x Multiplier.

Risk Category	Score Range	Multiplier	Underwriting Implication
LOW	0 – 25	1.0× (Standard)	Eligible for standard processing. No additional loading.
MEDIUM	26 – 50	1.2× (+20%)	Standard underwriter review required.
HIGH	51 – 75	1.5× (+50%)	Enhanced scrutiny required; senior underwriter preferred.
VERY HIGH	76 – 100	2.0× (+100%)	Mandatory senior sign-off. Special approval workflow.

6.2.1 Worked Calculation Example

Input Parameter	Value	Score Contribution
Base Monthly Premium	$500.00	N/A (base rate)
Parent Age	45 years (41–50 band)	+18 points
Occupation	Mining (Very High Hazard)	+25 points
Premium-to-Income Ratio	35% of monthly income	+20 points
Total Risk Score	**63 points**	**Category: HIGH**
Applied Multiplier	1.5×	+50% premium loading
Final Monthly Premium	**$750.00**	**Payable by Customer**

7 Underwriting Approval Workflow

Workflow	Underwriting Decision (Approve / Reject)
Actors	Underwriter, Admin
Objective	Provide a structured human review layer for policy applications, allowing qualified staff to verify documentation, review risk scores, adjust premiums, and render binding approval or rejection decisions.
Trigger	Underwriter logs in and navigates to the Assessment Queue in the staff portal.
APIs Involved	GET /api/underwriter/applications, GET /api/underwriter/applications/{id}, PUT /api/underwriter/applications/{id}/approve, PUT /api/underwriter/applications/{id}/reject
Backend Services	PolicyApplicationServiceImpl, EmailServiceImpl, AuditLogServiceImpl
Database Tables	SELECT policy_application WHERE status = PENDING; UPDATE policy_application SET status; INSERT policy_subscription; INSERT audit_logs
Initial State	Application in PENDING status. Risk score and adjusted premium already computed.
Final State	Application transitions to APPROVED (subscription created) or REJECTED (notification sent).

7.1 Approval Sub-flow

Step 1: Underwriter dispatches GET /api/underwriter/applications with optional filters (riskCategory, status). The paginated queue is ordered by submittedAt ascending (oldest first).

Step 2: Underwriter selects an application. GET /api/underwriter/applications/{id} returns the full detail: applicant profile, child details, risk breakdown, calculatedPremium, and a list of uploaded documents.

Step 3: Underwriter reviews the document completeness. The portal displays a mandatory document checklist per policy type. The Approve button is disabled until all mandatory documents are present.

▶️ **Decision Point:** Are all mandatory documents present? NO → Approve button remains disabled. Underwriter may request additional documents via the portal. YES → proceed.

Step 4: Underwriter optionally enters a revisedPremium in the approval form. This allows the underwriter to override the system-calculated premium based on their professional judgment.

Step 5: Underwriter dispatches PUT /api/underwriter/applications/{id}/approve with optional query parameter revisedPremium.

Step 6: PolicyApplicationServiceImpl.approveApplication() is invoked:

✔️ **Rule:** Application status must be PENDING. Any other status returns HTTP 409 ERR_APP_03.

✔️ **Rule:** If revisedPremium is supplied, it must be a positive value greater than zero.

Step 7: PolicyApplicationServiceImpl calls createSubscriptionForApprovedApplication(application). A PolicySubscription entity is created with: subscriptionNumber (SUB-XXXXXX), premiumAmount (revisedPremium or calculatedPremium), status = APPROVED, maturityDate calculated from startDate + durationYears.

Step 8: application.status is updated to APPROVED. application.decidedAt is set to now().

Step 9: AuditLogServiceImpl records APP_APPROVED with the underwriter's userId, the final premium, and any premium revision delta.

Step 10: EmailServiceImpl.sendApprovalEmail() generates a policy certificate PDF via PdfService (OpenPDF/iText) and sends it as an attachment to the customer.

7.2 Rejection Sub-flow

Step 1: Underwriter determines the application should be rejected (insufficient documentation, unacceptable risk, or fraud suspicion).

Step 2: Underwriter enters a rejectionReason in the portal rejection form.

▶️ **Decision Point:** Is rejectionReason blank? YES → HTTP 400 ERR_CLM_02 is returned immediately. NO → proceed.

Step 3: Underwriter dispatches PUT /api/underwriter/applications/{id}/reject?rejectionReason=<reason>.

Step 4: PolicyApplicationServiceImpl.rejectApplication() updates application.status to REJECTED and records the rejectionReason and decidedAt timestamp.

Step 5: AuditLogServiceImpl records APP_REJECTED with actor identity and stated reason.

Step 6: EmailServiceImpl.sendRejectionEmail() sends a notification to the customer containing the rejection reason and guidance on reapplication.

7.3 State Transitions

From State	Event	To State	Condition / Rule
PENDING	Underwriter approves	**APPROVED**	All documents present; risk acceptable; subscription created.
PENDING	Underwriter rejects	**REJECTED**	rejectionReason mandatory; customer notified.
APPROVED	First premium received	**ACTIVE**	Moves to Payment Activation workflow.
REJECTED	Customer reapplies	**PENDING**	New application record; previous rejection retained.

8 Policy Activation and Premium Payment Workflow

Workflow	Policy Activation and Premium Payment
Actors	Customer
Objective	Record premium payments against an approved subscription, activate coverage upon first payment, and maintain a running financial ledger for each policy subscription.
Trigger	Customer navigates to My Policies, selects an approved or active subscription, and clicks Pay Premium.
APIs Involved	POST /api/premium-payments, GET /api/premium-payments/history/{subscriptionId}
Backend Services	PremiumPaymentServiceImpl, AuditLogServiceImpl, EmailServiceImpl
Database Tables	INSERT premium_payment; UPDATE policy_subscription (status, totalPaidAmount, nextDueDate); INSERT audit_logs
Initial State	PolicySubscription in APPROVED status (first payment) or ACTIVE status (recurring). Payment amount provided by customer.
Final State	PremiumPayment record inserted. totalPaidAmount incremented. Subscription status set to ACTIVE (first payment). nextDueDate advanced.

8.1 First Payment — Subscription Activation

Step 1: Customer navigates to the My Policies dashboard. The subscription card shows status APPROVED and a Pay First Premium call-to-action.

Step 2: Customer confirms the payment amount (calculatedPremium or revisedPremium from underwriting).

Step 3: Frontend dispatches POST /api/premium-payments with PremiumPaymentRequest: { subscriptionId, amount, paymentDate }.

Step 4: PremiumPaymentServiceImpl.recordPayment() is invoked. The subscription is loaded via PolicySubscriptionRepository.findById(subscriptionId).

▶️ **Decision Point:** Is the subscription in APPROVED or ACTIVE status? NO (e.g., LAPSED or EXPIRED) → HTTP 400. YES → proceed.

Step 5: A PremiumPayment entity is created with status = COMPLETED and a system-generated receiptNumber (PAY-XXXXXX).

Step 6: PremiumPaymentRepository.save(payment) persists the record.

Step 7: subscription.totalPaidAmount += payment.amount.

✔️ **Rule:** If this is the first payment (totalPaidAmount was 0 before this payment), set subscription.status = ACTIVE and record the activatedAt timestamp.

Step 8: subscription.nextDueDate is advanced by one payment frequency period (e.g., +1 month for MONTHLY).

Step 9: PolicySubscriptionRepository.save(subscription) persists the updated state.

Step 10: AuditLogServiceImpl records PREMIUM_PAID with amount, subscriptionId, and actor.

Step 11: EmailServiceImpl dispatches a payment receipt email to the customer.

8.2 Recurring Payment

For recurring payments after activation, Steps 3 through 11 above apply identically, with the following difference: subscription.status remains ACTIVE and is not re-activated. The totalPaidAmount continues to accumulate, and the maturity benefit calculation is based on this aggregate.

> ⚠️ **Exception:** If a payment is missed and the grace period (30 days) expires, the system's scheduled maturity check job sets subscription.status = LAPSED. A lapsed subscription can be reinstated by making the overdue payment plus the applicable reinstatement fee.

8.3 State Transitions

From State	Event	To State	Condition / Rule
APPROVED	First premium payment received	**ACTIVE**	subscription.status set to ACTIVE; coverage begins.
ACTIVE	Recurring payment received	**ACTIVE**	totalPaidAmount incremented; nextDueDate advanced.
ACTIVE	Grace period (30 days) expires	**LAPSED**	Scheduled job triggers; customer notified.
LAPSED	Reinstatement payment received	**ACTIVE**	Overdue premium + reinstatement fee paid.
ACTIVE	Maturity date reached	**EXPIRED**	Policy term complete; payout triggered.

9 Claim Submission Workflow

Workflow	Claim Submission
Actors	Customer
Objective	Allow a policyholder to initiate a benefit claim against an active subscription and enter the claim into the Claims Officer review queue.
Trigger	Customer navigates to My Claims and selects Raise New Claim, either at policy maturity or upon a qualifying contingency event.
APIs Involved	POST /api/claims, POST /api/documents/upload (claim evidence)
Backend Services	ClaimServiceImpl, AuditLogServiceImpl, EmailServiceImpl
Database Tables	SELECT policy_subscription; SELECT claim (for remainingCoverage); INSERT claim; INSERT audit_logs
Initial State	PolicySubscription in ACTIVE status. Customer has supporting documentation available.
Final State	Claim record saved with status SUBMITTED. Claim appears in the Claims Officer queue.

9.1 Step-by-Step Process

Step 1: Customer selects an active subscription from their dashboard and clicks Raise New Claim.

Step 2: Customer fills the ClaimRequest form: claimType (e.g., TUITION_FEE, MEDICAL), claimAmount, claimDate.

Step 3: Frontend dispatches POST /api/claims with the ClaimRequest payload.

Step 4: ClaimServiceImpl.raiseClaim() is invoked.

Step 5: ClaimServiceImpl loads the subscription and verifies its status.

▶️ **Decision Point:** Is the subscription ACTIVE? NO → HTTP 400 ERR_CLM_03. YES → proceed.

Step 6: ClaimServiceImpl calculates remainingCoverage: policy.riskCoverageAmount minus the sum of all previously APPROVED or PAID claims against this subscription.

▶️ **Decision Point:** Is claimAmount > remainingCoverage? YES → HTTP 422 ERR_CLM_01. NO → proceed.

Step 7: A Claim entity is created with status = SUBMITTED and claimDate.

Step 8: ClaimRepository.save(claim) persists the record.

Step 9: AuditLogServiceImpl records CLAIM_SUBMITTED.

Step 10: Customer is prompted to upload supporting documents via POST /api/documents/upload with the claimId in the request parameters.

Step 11: A submission acknowledgement email is dispatched to the customer asynchronously.

9.2 Validation Rules

Rule	Specification
Active Subscription	Claim can only be raised against a subscription with status = ACTIVE.

Amount Ceiling	claimAmount must not exceed the remainingCoverage (riskCoverageAmount minus prior approved claims).
Ownership	The authenticated user must be the owner of the referenced subscription.
Date Validity	claimDate must not be in the future.

10 Claim Verification and Approval Workflow

Workflow	Claim Verification and Approval
Actors	Claims Officer, Admin
Objective	Verify the legitimacy and accuracy of a submitted claim against policy terms and uploaded evidence, then authorize or reject the payout.
Trigger	Claims Officer logs in and views the incoming claims queue via GET /api/claims-officer/claims.
APIs Involved	GET /api/claims-officer/claims, GET /api/claims-officer/claims/{id}, PUT /api/claims-officer/claims/{id}/approve, PUT /api/claims-officer/claims/{id}/reject
Backend Services	ClaimServiceImpl, AuditLogServiceImpl, EmailServiceImpl
Database Tables	SELECT claim; SELECT documents (for evidence); UPDATE claim SET status, payout_date; INSERT audit_logs
Initial State	Claim in SUBMITTED status. Supporting documents uploaded by customer.
Final State	Claim transitions to APPROVED with payoutDate set, or REJECTED with stated reason.

10.1 Verification Sub-flow

Step 1: Claims Officer dispatches GET /api/claims-officer/claims (paginated, filterable by status). Claims are ordered by claimDate ascending.

Step 2: Officer selects a claim. The detail view (ClaimVerifierComponent) renders a side-by-side display: claimed amount on the left; downloaded documents on the right.

Step 3: Officer reviews the uploaded evidence against the claim type and amount. The subscription's totalPaidAmount and riskCoverageAmount are displayed for context.

Step 4: Officer verifies that the claimed amount does not exceed the policy's riskCoverageAmount at approval time.

▶️ **Decision Point:** Does the evidence support the claim? YES → proceed to approval. NO → proceed to rejection.

10.2 Approval Sub-flow

Step 5: Officer dispatches PUT /api/claims-officer/claims/{id}/approve.

Step 6: ClaimServiceImpl.approveClaim() verifies claim.status is SUBMITTED or UNDER_REVIEW.

Step 7: claim.status is set to APPROVED. claim.payoutDate is set to LocalDate.now().plusDays(7) (T+7 business days from approval).

Step 8: A disbursement reference number (DISB-YYYY-XXXXXX) is generated and stored.

Step 9: AuditLogServiceImpl records CLAIM_APPROVED with officer identity, approved amount, and payoutDate.

Step 10: EmailServiceImpl sends an approval notification to the customer including the payoutDate and disbursement reference.

10.3 Rejection Sub-flow

Step 5: Officer enters a mandatory rejectionReason in the rejection form.

Step 6: Officer dispatches PUT /api/claims-officer/claims/{id}/reject with rejectionReason.

▶ **Decision Point:** Is rejectionReason blank? YES → HTTP 400 ERR_CLM_02. NO → proceed.

Step 7: claim.status set to REJECTED. rejectionReason stored.

Step 8: AuditLogServiceImpl records CLAIM_REJECTED with reason.

Step 9: EmailServiceImpl sends a rejection notification with the reason and appeal instructions.

10.4 State Transitions

From State	Event	To State	Condition / Rule
SUBMITTED	Claims Officer assigns review	**UNDER_REVIEW**	Officer accepts claim for investigation.
SUBMITTED / UNDER_REVIEW	Officer approves	**APPROVED**	payoutDate = today + 7 days; customer notified.
APPROVED	Finance disburses	**PAID**	Payment dispatched; subscription updated.
SUBMITTED / UNDER_REVIEW	Officer rejects	**REJECTED**	rejectionReason mandatory; customer notified.
REJECTED	Customer resubmits with corrections	**SUBMITTED**	New submission referencing same subscription.

11 Document Upload and Verification Workflow

Workflow	Document Upload and Verification
Actors	Customer (upload), Underwriter (download and review), Claims Officer (download and review)
Objective	Provide a secure mechanism for uploading, storing, and retrieving supporting documents at both the policy application and claim stages.
Trigger	During policy application wizard (document upload step) or after claim submission (evidence attachment).
APIs Involved	POST /api/documents/upload, GET /api/documents/download/{documentId}
Backend Services	DocumentServiceImpl
Database Tables	INSERT document; SELECT document; SELECT by applicationId or claimId
Initial State	A valid policy application ID or claim ID exists. Customer has the document file ready.
Final State	Document metadata persisted in the documents table. Binary file stored on server filesystem.

11.1 Upload Process

Step 1: Customer uses the Angular file picker to select a file. The component validates the MIME type client-side (PDF, JPG, PNG only) before dispatching the request.

Step 2: Frontend dispatches POST /api/documents/upload as multipart/form-data with: file (binary), applicationId or claimId, documentType, userId.

Step 3: DocumentServiceImpl.uploadDocument() is invoked.

Step 4: Server-side MIME type validation: the Content-Type of the uploaded file is verified against the permitted list (application/pdf, image/jpeg, image/png).

▶ **Decision Point:** Is file type permitted? NO → HTTP 415 ERR_DOC_04. YES → proceed.

Step 5: File size is checked against the 10 MB maximum limit.

▶ **Decision Point:** Does file exceed 10 MB? YES → HTTP 413 ERR_DOC_05. NO → proceed.

Step 6: DocumentServiceImpl generates a UUID-based storage filename to prevent collisions and directory traversal attacks. The original filename is retained in metadata only.

Step 7: The binary is written to the configured server storage path.

Step 8: A Document entity is persisted: documentType, originalFilename, storagePath, mimeType, fileSizeBytes, applicationId or claimId, uploadedBy, uploadedAt.

Step 9: Response: { documentId, documentType, originalFilename, uploadedAt }.

11.2 Download / Review Process

Step 1: Underwriter or Claims Officer views the application or claim detail page. The portal lists all associated documents by type.

Step 2: Staff member clicks a document link. The Angular component dispatches GET /api/documents/download/{documentId}.

Step 3: DocumentServiceImpl loads the Document metadata and reads the binary from the storage path.

Step 4: The file is streamed as a binary response with Content-Type and Content-Disposition headers matching the original upload.

11.3 Business and Validation Rules

✔ **Rule:** Only PDF, JPEG, and PNG file types are accepted. Executables, scripts, and archives are rejected.

✔ **Rule:** Maximum individual file size: 10 MB.

✔ **Rule:** Documents are stored with UUID-based filenames. The original filename is preserved in the documents table for display purposes only.

✔ **Rule:** Uploading a document to a REJECTED application is blocked at the service layer.

✔ **Rule:** Document access control: customers may only upload to their own applications/claims. Downloads are accessible to authorized staff roles.

12 Email Notification Workflow

Workflow	Asynchronous Email Notification
Actors	System (triggered by service layer events)
Objective	Ensure all relevant lifecycle events are communicated to affected users via templated, professional email notifications dispatched asynchronously.
Trigger	Any state-changing service method that affects a customer or staff actor (e.g., approveApplication, approveClaim, registerUser).
APIs Involved	No direct external API. EmailServiceImpl is invoked internally by PolicyApplicationServiceImpl, ClaimServiceImpl, and AuthServiceImpl.
Backend Services	EmailServiceImpl, JavaMailSender, Thymeleaf Template Engine
Database Tables	No direct database write. Reads user.email for addressing.
Initial State	A lifecycle event has occurred in another service.
Final State	Email dispatched to the affected user's registered email address. Delivery confirmation logged.

12.1 Notification Trigger Map

Triggering Event	Recipient	Email Content Summary
User Registration	New Customer	Welcome message, account confirmation, login link, and platform overview.
Password Reset Request	Account Owner	6-digit OTP valid for 10 minutes with reset instructions.
Application Submitted	Customer	Acknowledgement with policyNumber, riskCategory, and expected review timeline.
Application Approved	Customer	Approval confirmation with attached PDF policy certificate, premium amount, and subscription number.
Application Rejected	Customer	Rejection notification with stated reason and reapplication guidance.
Premium Payment Received	Customer	Payment receipt with amount, date, receipt number, and updated totalPaidAmount.
Claim Submitted	Customer	Submission acknowledgement with claimId and expected processing timeline.
Claim Approved	Customer	Approval notification with payoutDate (T+7) and disbursement reference number.
Claim Rejected	Customer	Rejection notification with reason and appeal instructions.
Subscription Lapsed	Customer	Lapse warning with reinstatement options and grace

		period information.

12.2 Email Construction Process

Step 1: Triggering service (e.g., PolicyApplicationServiceImpl) calls emailService.sendApplicationStatusEmail(user, application).

Step 2: EmailServiceImpl constructs a Thymeleaf Context object, populating template variables: userName, policyNumber, status, premium, dates, etc.

Step 3: For approval emails, PdfService.generateCertificate(application) is called to produce a byte array PDF using OpenPDF/iText.

Step 4: A MimeMessage is constructed using JavaMailSender. For certificate emails, the PDF is attached as a multipart/mixed MIME part.

Step 5: JavaMailSender.send(message) dispatches the email via the configured SMTP relay.

✔ **Rule:** Email dispatch is asynchronous (@Async annotation on EmailServiceImpl methods). A failure to send email does NOT roll back the primary business transaction.

13 Audit Logging Workflow

Workflow	System-Wide Audit Logging
Actors	System (automatic, triggered by service layer)
Objective	Maintain a complete, tamper-evident, append-only record of all significant user actions and system state transitions for compliance, forensic investigation, and regulatory review.
Trigger	Any state-changing API call processed by the service layer. Audit entries are created after the primary operation completes successfully.
APIs Involved	(Write) Internal call from service layer. (Read) GET /api/admin/audit-logs (ADMIN only).
Backend Services	AuditLogServiceImpl
Database Tables	INSERT audit_logs (append-only). No UPDATE or DELETE operations on this table.
Initial State	A business operation has completed successfully.
Final State	Immutable audit record persisted with actor identity, action code, entity reference, and timestamp.

13.1 Logged Event Categories

Category	Action Codes	Description
User Events	LOGIN FAILED_LOGIN LOGOUT PASSWORD_CHANGE ACCOUNT_LOCKED	All authentication events including failures, for security monitoring.
Application Events	CREATE_APPLICATION CANCEL_APPLICATION APP_APPROVED APP_REJECTED	Complete application lifecycle from submission to decision.
Claim Events	CLAIM_SUBMITTED CLAIM_APPROVED CLAIM_REJECTED CLAIM_PAID	Full claim lifecycle from submission to payout.
Financial Events	PREMIUM_PAID SUBSCRIPTION_ACTIVATED SUBSCRIPTION_LAPSED	All financial transactions and subscription status changes.
Admin Events	USER_DEACTIVATED POLICY_CREATED POLICY_DEACTIVATED STAFF_CREATED	Platform governance actions taken by administrators.

Document Events	DOCUMENT_UPLOADED DOCUMENT_DOWNLOADED	Evidence management tracking.

13.2 Audit Record Structure

Field	Type	Description
logId	BIGINT (PK)	Auto-incremented surrogate key.
action	VARCHAR(80)	Action code from the defined enum (e.g., APP_APPROVED).
actorEmail	VARCHAR(100)	Email of the user who triggered the event.
actorRole	VARCHAR(30)	Role of the actor at the time of the event.
entityType	VARCHAR(50)	Affected entity class name (e.g., PolicyApplication).
entityId	BIGINT	Primary key of the affected record.
detail	TEXT	Human-readable description of the change (e.g., Premium revised from 500 to 650).
ipAddress	VARCHAR(45)	Client IP address if available from the HTTP request context.
createdAt	TIMESTAMP	Immutable creation timestamp. Set by the database DEFAULT NOW().

✔ **Rule:** The audit_logs table has no UPDATE or DELETE permissions granted at the database level. Records are append-only. This immutability is a regulatory requirement.

✔ **Rule:** Audit log retrieval is restricted to the ADMIN role exclusively via GET /api/admin/audit-logs, which supports filtering by action, actorEmail, startDate, and endDate.

14 Admin Management Workflow

Workflow	Admin Platform Management
Actors	Admin
Objective	Provide comprehensive platform governance tools including user management, policy product configuration, and business analytics.
Trigger	Admin logs in to the administrative dashboard.
APIs Involved	GET /api/users, PUT /api/users/{id}/status, POST /api/admin/policies, PUT /api/admin/policies/{id}/deactivate, GET /api/admin/dashboard/overview, GET /api/admin/dashboard/revenue, GET /api/admin/audit-logs
Backend Services	UserServiceImpl, PolicyServiceImpl, AnalyticsServiceImpl, AuditLogServiceImpl
Database Tables	SELECT / UPDATE users; UPDATE policy_types; SELECT aggregations for analytics
Initial State	Admin is authenticated with ROLE_ADMIN.
Final State	Platform state updated per administrative action. All changes audit-logged.

14.1 User Management

Step 1: Admin navigates to User Management. GET /api/users returns a paginated, filterable list of all accounts (all roles).

Step 2: Admin can view individual user details, associated applications, and subscription status.

Step 3: Admin toggles a user's active status via PUT /api/users/{id}/status. Setting isActive = false immediately invalidates the user's ability to authenticate. Existing tokens are rejected at the Spring Security filter chain on the next request.

✔ **Rule:** User deactivation is a soft operation. No data is deleted. The account is anonymized only upon a formal GDPR deletion request.

▶ **Decision Point:** Is the target user the only ADMIN account? YES → Deactivation is blocked to prevent administrative lockout. NO → proceed.

14.2 Policy Product Management

Step 1: Admin navigates to Policy Configuration. GET /api/admin/policies lists all products including discontinued ones.

Step 2: Admin creates a new product via POST /api/admin/policies with: policyName, basePremium, durationYears, minChildAge, maxChildAge, riskCoverageAmount.

Step 3: Admin discontinues an existing product via PUT /api/admin/policies/{id}/deactivate. This sets isActive = false.

✔ **Rule:** Discontinuation is a soft operation. Existing subscriptions under a discontinued policy are unaffected and continue to operate normally.

✔ **Rule:** Changes to basePremium on an existing policy apply to new applications only. Active subscriptions retain their contracted premium.

14.3 Dashboard Analytics

Step 1: Admin navigates to Dashboard Overview. GET /api/admin/dashboard/overview triggers AnalyticsServiceImpl.

Step 2: AnalyticsServiceImpl executes optimized JPA queries annotated with @Transactional(readOnly = true):

- countUsers() — total registered accounts grouped by role.
- calculateTotalPremium() — sum of all COMPLETED premium payments.
- getPolicyDistribution() — breakdown of applications by status and policy type.
- getClaimApprovalRate() — ratio of APPROVED claims to total submitted.

Step 3: GET /api/admin/dashboard/revenue returns findMonthlyRevenue() results: an array of { month, year, totalPremiumCollected, totalClaimsPaid } objects for trend visualization.

15 Staff Management Workflow

Workflow	Staff Provisioning and Management
Actors	Admin
Objective	Enable administrators to create and manage Underwriter and Claims Officer accounts, controlling access to the EduShield staff portals.
Trigger	Admin navigates to Team Management and clicks Add Staff Member.
APIs Involved	POST /api/admin/create-staff, GET /api/users (filtered by role), PUT /api/users/{id}/status
Backend Services	AuthServiceImpl, AdminUserController, AuditLogServiceImpl
Database Tables	INSERT users (with staff role); SELECT roles; INSERT audit_logs
Initial State	Admin is authenticated. Target email does not already exist in the system.
Final State	Staff account created with the assigned role. Account is immediately active.

15.1 Staff Creation Process

Step 1: Admin fills the Staff Creation form: email, temporaryPassword, role (UNDERWRITER or CLAIMS_OFFICER), fullName.

Step 2: Admin dispatches POST /api/admin/create-staff with the StaffCreateRequest payload.

Step 3: AdminUserController passes the request to AuthServiceImpl.createStaffAccount().

Step 4: AuthServiceImpl checks that the email does not already exist via UserRepository.existsByEmail(). Duplicate returns HTTP 400.

Step 5: The specified role is loaded from RoleRepository. If the role is not UNDERWRITER or CLAIMS_OFFICER, an authorization exception is thrown — staff creation cannot elevate a user to ADMIN.

Step 6: BCryptPasswordEncoder encodes the temporary password.

Step 7: User entity created with isActive = true. UserRepository.save(user) persists the record.

Step 8: AuditLogServiceImpl records STAFF_CREATED with the admin's identity and the new staff member's email and role.

Step 9: EmailServiceImpl sends a Welcome Email to the new staff member with their login credentials and a mandatory password-change notice.

15.2 Staff Access Control

Staff Role	Permitted Actions	Restricted From
UNDERWRITER	View application queue Review risk scores Approve or reject applications View customer documents Download policy certificates	Customer data modification Claim processing Admin configuration Audit log access
CLAIMS_OFFICER	View claims queue	Application underwriting

	Verify claim documents Approve or reject claims View subscription details	Policy configuration Admin management Audit log access

16 System Integration Workflow

This section documents how the internal services of EduShield interact with each other and with external systems during normal operation. Understanding inter-service dependencies is essential for debugging, performance optimization, and future scalability planning.

16.1 Inter-Service Communication Map

Calling Service	Called Service	Context / Trigger
PolicyApplicationServiceImpl	RiskAssessmentServiceImpl	Called synchronously during applyForPolicy(). Result (riskScore, category, multiplier) is used immediately.
PolicyApplicationServiceImpl	EmailServiceImpl	Called asynchronously after application submission, approval, or rejection.
PolicyApplicationServiceImpl	AuditLogServiceImpl	Called synchronously after every state change in the application lifecycle.
PolicyApplicationServiceImpl	PdfService	Called during approval to generate the policy certificate PDF for the email attachment.
ClaimServiceImpl	EmailServiceImpl	Called asynchronously after claim submission, approval, or rejection.
ClaimServiceImpl	AuditLogServiceImpl	Called synchronously after every claim state transition.
PremiumPaymentServiceImpl	AuditLogServiceImpl	Called synchronously after every successful payment recording.
AuthServiceImpl	JwtTokenProvider	Called to generate JWT after successful registration or login.
GoogleAuthServiceImpl	GoogleIdTokenVerifier	Called to validate Google OAuth2 tokens against Google's public key infrastructure.
EmailServiceImpl	JavaMailSender (SMTP)	All outbound email dispatched via configured SMTP relay (external dependency).
AnalyticsServiceImpl	All Repositories (readOnly)	Dashboard analytics queries use @Transactional(readOnly=true) for performance.

16.2 Full End-to-End Flow Summary

The following table presents the complete customer journey from initial registration through policy maturity, showing the sequence of workflows, services, and state transitions.

#	Phase	Workflow	Outcome
1	Onboarding	User Authentication	Account created. JWT issued. Session active.
2	Discovery	Policy Discovery	Customer selects insurance product.

3	Application	Policy Application	Application saved as PENDING. Risk score computed.
4	Automated	Risk Assessment	Risk category and adjusted premium determined.
5	Review	Underwriting	Application APPROVED / REJECTED. Subscription created.
6	Activation	Premium Payment	First payment received. Subscription ACTIVE.
7	Maintenance	Recurring Payment	Monthly premiums collected. totalPaidAmount grows.
8	Claim	Claim Submission	Claim SUBMITTED. Evidence uploaded.
9	Verification	Claim Approval	Claim APPROVED. PayoutDate = T+7 days.
10	Closure	Payout	Claim PAID. Subscription record closed.

16.3 External System Dependencies

External System	Integration Point	Failure Impact
SMTP Mail Server	JavaMailSender / EmailServiceImpl	Email delivery fails silently. Business transactions are not rolled back. Emails may be retried via queue (future scope).
Google OAuth2	GoogleIdTokenVerifier	Google login unavailable. Local email/password login remains functional as a fallback.
File Storage (filesystem / S3)	DocumentServiceImpl	Document upload and download unavailable. Application submission may proceed without documents (flagged as DOCUMENTATION_PENDING).
PostgreSQL Database	All JPA Repositories	Complete system unavailability. All API endpoints return 503. ACID guarantees prevent partial writes.

[END OF SYSTEM WORKFLOW DOCUMENTATION v1.0]

EduShield – Prepared by Praveen Vankudoth | March 2026 | CONFIDENTIAL